

Question 1: Heuristic design and admissibility [15 points]

For both the frequency and step heuristics, the value returned was just the amount of mismatched letters in the current state, to the goal state.

For scrabble, whenever there is a mismatch character I use the scrabble points table to add that character's point value to the cost. Then after every letter the total was the value returned.

Frequency + Step Heuristics:

```
for each letter in state:
    If current[i] != goal[i]:
        cost += 1
return cost
```

The unavoidable costs are at least the amount of mismatched characters, since that is the amount that needs to change regardless.

It shouldn't overestimate, since it only estimates the current step to the final step, not accounting for the intermediate transition steps.

Scrabble:

```
for each letter in state:
    If current[i] != goal[i]:
        for value, letters in scrabblelist:
            If goal[i] in letters:
                cost += value
                break
return cost
```

The unavoidable costs are that each mismatched letter will eventually need to reach the final state, so the total cost will need to change regardless.

Shouldn't overestimate since it counts the cost of inserting each required goal letter only once. If the solution needs extra intermediate steps, those add additional cost. So the cheapest remaining cost can be greater than the heuristic but not less.

Question 2: Does the heuristic actually help? [10 points]

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY->ASK	Steps	5	773	5	16	97.9%
WHY->ASK	Scrabble	9	765	9	13	98.3%
WHY->ASK	Frequency	38.85	808	38.85	755	6.6%
BEAR -> DUCK	Steps	4	691	4	6	99.1%
BEAR -> DUCK	Scrabble	11	2981	11	6	99.8%
BEAR -> DUCK	Frequency	38.24	1084	38.24	720	33.6%

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yes, both returned the same cost

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

Scrabble

3. For each word pair, under which cost measure was the heuristic least informative?

Frequency

4. Explain the differences using the definitions of your heuristics, not runtime alone.

Both frequency and steps use the same heuristic to count the number of mismatch characters, while scrabble counts the score of each mismatched letter, not just the occurrence of a non matching letter.

Both steps and scrabble heuristics accurately reflected the steps and final states.

Frequency I think was hit the worst since it used the same heuristic as steps, but each word actually has a score value that wasn't used.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

1. State yes or no.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the provided dictionary.

Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Steps: Yes, since the amount of intermediate steps shouldn't change, so the cost is not changing

Scrabble: No, because the letter that replaced the current letter can be different which results in a different score.

Bear $\rightarrow$ Dear, d cost = 2

Dear $\rightarrow$ Bear, b cost = 3

Frequency: No, since this is reliant on the current word score, which can be different

Bear $\rightarrow$ Dear, cost = $1 + \text{rarity}(\text{dear})$

Dear $\rightarrow$ Bear, cost = $1 + \text{rarity}(\text{bear})$

Different rarity values for those words

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

I used gemini with the prompt above. For any non-goal state, the proposed heuristic $h_R(n)$ adds the goal word's rarity value plus the minimum possible dictionary rarity for each intermediate step to the baseline Hamming distance $h_0(n)$.

2. A precise formula or pseudocode for the proposed heuristic.

```
minWord = min(for word in dictionary.get(word.rarity))
```

```
function count_different_letters(curr, goal):  
    mismatches = 0  
    for i from 0 to length(word) - 1:  
        if curr[i] != goal[i]:  
            mismatches = mismatches + 1  
    return mismatches
```

```

function h(current, goal, minWord):
    if current == goal:
        return 0
    h0 = count_different_letters(current, goal)
    return h0 + goal.rarity + (h0 - 1) * minWord

```

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

The proposed heuristic $h_R(n)$ is admissible because $h_R(n) \leq h^*(n)$ for every state n , meaning it never overestimates the true optimal cost to reach the goal state g .

Any valid path from state n to goal g requires at least $h_0(n)$ steps because each legal move changes only one letter at a time.

The total cost of that path is the sum of step costs plus the goal's rarity and the rarities of all intermediate words, each of which is at least $R_{\min}$.

Because $h_R(n)$ calculates this exact absolute minimum cost using $h_0(n)$ base steps, the goal's rarity, and $(h_0(n) - 1) * R_{\min}$ for intermediate steps, it will never exceed the true optimal cost $h^*(n)$.

4. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Word Pair	Cost Measure	Baseline h_0 Solution Cost	Baseline h_0 Expanded Nodes	Proposed h_R Solution Cost	Proposed h_R Expanded Nodes (Expected)	Performance Impact
WHY -> ASK	Frequency	38.85	755	38.85	755	Less nodes

BEAR -> DUCK	Frequenc y	38.24	720	38.24	720	Less nodes
-----------------	---------------	-------	-----	-------	-----	------------

The experimental results support the proposed heuristic because assigning more accurate estimates to graph nodes, like raising WHY from 3 to 10.0 and WAY from 2 to 8.0, allows A* to eradicate bad branches earlier while keeping the best solution costs.